Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-207БВ-25

Студент: Горбунов А.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 07.11.26

Москва, 2026

Постановка задачи

Вариант 5.

Пользователь вводит команды вида: «число<endline>». Далее это число передается от родительского процесса в дочерний. Дочерний процесс производит проверку на простоту. Если число составное, то в это число записывается в файл. Если число отрицательное или простое, то тогда дочерний и родительский процессы завершаются.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void) – создает дочерний процесс, который является полной копией родительского (разделяет код, но имеет собственное адресное пространство).
- int pipe(int *fd) – создает неименованный канал для однонаправленной передачи данных. Возвращает два файловых дескриптора: fd[0] (для чтения) и fd[1] (для записи).
- int dup2(int oldfd, int newfd) – перенаправляет файловые дескрипторы. В данной работе используется для подмены стандартного ввода и вывода дочернего процесса на каналы.
- int execl(const char *path, const char *arg, ...) – заменяет текущий образ памяти процесса на новую программу. Используется для запуска программы child из процесса-родителя.
- int open(const char *pathname, int flags, mode_t mode) – открывает или создает файл. В работе применяются флаги O_WRONLY, O_CREAT, O_TRUNC для создания нового файла или очистки существующего.
- ssize_t read(int fd, void *buf, size_t count) и ssize_t write(int fd, const void *buf, size_t count) – используются для чтения и записи данных в файловые дескрипторы (каналы и файлы), так как использование стандартной библиотеки stdio.h было запрещено.
- pid_t wait(int *wstatus) – приостанавливает выполнение родительского процесса до завершения дочернего, предотвращая появление процессов-зомби.
- int close(int fd) – закрывает файловый дескриптор, освобождая ресурсы.

Программа состоит из двух исполняемых файлов, представляющих разные процессы. Родительский процесс parent.c запрашивает у пользователя имя файла, создает два канала pipe1 и pipe2 для организации двусторонней связи и с помощью fork() порождает дочерний процесс. С помощью dup2 дочерний процесс перенаправляет свой стандартный ввод (stdin) в pipe1, а стандартный вывод (stdout) – в pipe2. Затем с помощью execl он запускает программу child, передавая ей имя файла аргументом.

Родительский процесс в цикле считывает введенные числа и отправляет их в pipe1. Дочерний процесс child.c читает числа, проверяет их на простоту. Если число составное, оно записывается в файл и дублируется в pipe2 (для вывода на экран). Если число отрицательное или простое – дочерний процесс пишет в pipe2 строку "TERMINATE" и завершается. Родитель, прочитав эту строку, прерывает цикл, закрывает дескрипторы и завершается. Все критические системные вызовы (pipe, fork, read, write, open) проверяются на возникновение ошибок.

Код программы

parent.c

```
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int starts_with_terminate(const char *str) {
    const char *prefix = "TERMINATE";
    while (*prefix) {
        if (*str != *prefix) return 0;
        str++; prefix++;
    }
    return 1;
}

int main() {
    char file_name[256];
    ssize_t file_name_len = read(STDIN_FILENO, file_name, sizeof(file_name) - 1);
    if (file_name_len <= 0) {
        const char msg[] = "error: failed to read file name\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    if (file_name[file_name_len - 1] == '\n') {
        file_name[file_name_len - 1] = '\0';
    } else {
        file_name[file_name_len] = '\0';
    }

    int parent_to_child[2];
    int child_to_parent[2];
```

```
if (pipe(parent_to_child) == -1 || pipe(child_to_parent) == -1) {
    const char msg[] = "error: pipe failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

pid_t pid = fork();
if (pid == -1) {
    const char msg[] = "error: fork failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

if (pid == 0) {
    close(parent_to_child[1]);
    close(child_to_parent[0]);

    dup2(parent_to_child[0], STDIN_FILENO);
    close(parent_to_child[0]);

    dup2(child_to_parent[1], STDOUT_FILENO);
    close(child_to_parent[1]);

    execl("./child", "child", file_name, NULL);

    const char msg[] = "error: execl failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

close(parent_to_child[0]);
close(child_to_parent[1]);
```

```
char buffer[1024];
ssize_t bytes = 0;
ssize_t written = 0;

while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0) {
    if (buffer[0] == '\n') break;

    written = write(parent_to_child[1], buffer, bytes);
    if (written != bytes) {
        const char msg[] = "error: failed to write to parent_to_child\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    bytes = read(child_to_parent[0], buffer, sizeof(buffer) - 1);
    if (bytes < 0) {
        const char msg[] = "error: failed to read from child_to_parent\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    if (bytes == 0) break;

    buffer[bytes] = '\0';

    if (starts_with_terminate(buffer)) {
        break;
    }

    written = write(STDOUT_FILENO, buffer, bytes);
    if (written != bytes) {
        const char msg[] = "error: failed to write to stdout\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
```

```
        }
    }

    close(parent_to_child[1]);
    close(child_to_parent[0]);

    wait(NULL);
    return 0;
}
```

child.c

```
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>

long long my_atoll(const char *s) {
    long long res = 0;
    int sign = 1;
    while (*s == ' ' || *s == '\t' || *s == '\n') s++;
    if (*s == '-') { sign = -1; s++; }
    while (*s >= '0' && *s <= '9') {
        res = res * 10 + (*s - '0');
        s++;
    }
    return res * sign;
}

int is_prime(long long num) {
    if (num < 2) return 0;
    for (long long i = 2; i * i <= num; i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}
```

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        const char msg[] = "error: file path not provided\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        const char msg[] = "error: failed to open target file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    char buffer[1024];
    ssize_t bytes = 0;
    ssize_t written = 0;

    while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[bytes] = '\0';
        long long num = my_atoll(buffer);

        if (num < 0 || is_prime(num)) {
            const char msg[] = "TERMINATE\n";
            write(STDOUT_FILENO, msg, sizeof(msg) - 1);
            close(fd);
            exit(EXIT_SUCCESS);
        }

        written = write(fd, buffer, bytes);
        if (written != bytes) {
            const char msg[] = "error: failed to write to file\n";
```

```
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            close(fd);
            exit(EXIT_FAILURE);
        }

        written = write(STDOUT_FILENO, buffer, bytes);
        if (written != bytes) {
            const char msg[] = "error: failed to write to stdout\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            close(fd);
            exit(EXIT_FAILURE);
        }
    }

    close(fd);
    return 0;
}
```

Протокол работы программы

```
virex@Ubuntu-OS:~/os-labs/lab1$ make
gcc parent.c -o parent
gcc child.c -o child
virex@Ubuntu-OS:~/os-labs/lab1$ ./parent
test1.txt
4
4
8
8
20
20
7
virex@Ubuntu-OS:~/os-labs/lab1$ cat test1.txt
4
```

```
8
20
virex@Ubuntu-OS:~/os-labs/lab1$ ./parent
test2.txt
-5
virex@Ubuntu-OS:~/os-labs/lab1$ cat test2.txt
virex@Ubuntu-OS:~/os-labs/lab1$ ./parent
test3.txt
55
55
14
14
25
25
-6
virex@Ubuntu-OS:~/os-labs/lab1$ cat test3.txt
55
14
25
```

Вывод

В ходе лабораторной работы я получил практические навыки создания процессов в Linux с помощью fork() и execl(), а также организации их взаимодействия через каналы pipe. Главной сложностью стало освоение механизма перенаправления дескрипторов (dup2) и написание собственного парсера чисел из-за запрета на библиотеку stdio.h. В результате программа успешно работает согласно заданию, корректно обрабатывает системные ошибки и завершается по сигналу от дочернего процесса.